

FIN 580 - Final Project

Abby Pham, Hannah Choi, Ram Narayanan

April 2026

Link to GitHub:

<https://github.com/hannahchoi05/forest-through-the-trees>

1 Abstract

This paper replicates and extends the framework of (Bryzgalova et al.), which introduces Asset Pricing Trees (AP-Trees) as a flexible alternative to traditional sorting-based methods for constructing cross-sections of stock returns. While conventional double and triple sorting impose coarse, unconditional partitions that fail to capture nonlinear interactions, AP-Trees use sequential conditional splits and global pruning to construct more informative basis assets for spanning the stochastic discount factor (SDF). We first confirm that AP-Trees outperform traditional sorting methods in terms of Sharpe ratios and diversification, and then extend the framework to an implementable trading setting by incorporating transaction costs, turnover, and dynamic portfolio construction. We further examine a residual momentum augmentation to enhance cross-sectional return predictability. Our results show that although AP-Trees provide a richer representation of the cross-section, their strong statistical performance does not directly translate into implementable strategies without accounting for trading frictions. Incorporating transaction costs reduces returns but improves realism, while signal-based enhancements introduce a trade-off between performance and turnover, highlighting the importance of jointly evaluating asset pricing models and portfolio construction constraints.

2 Introduction

A central challenge in empirical asset pricing is constructing portfolios that capture the information embedded in firm characteristics while remaining interpretable and practical. Traditional double and triple sorting methods, while widely used, impose coarse, unconditional partitions that fail to capture nonlinear interactions and the joint distribution of characteristics, resulting in an incomplete representation of expected returns. These limitations worsen as the number of characteristics increases, leading to the curse of dimensionality, where portfolios become sparse, redundant, or poorly diversified. This raises a

key question: how can we construct a low-dimensional yet informative set of portfolios that fully reflects the underlying structure of the data?

(Bryzgalova et al.) address this problem with Asset Pricing Trees (AP-Trees), which replace unconditional sorting with sequential conditional splits and apply global pruning to select a sparse set of portfolios that span the stochastic discount factor (SDF). While this framework improves statistical performance, it is primarily designed for asset pricing evaluation and does not account for practical implementation constraints such as transaction costs, turnover, and portfolio stability. In this paper, we replicate the AP-Trees framework and extend it to a realistic trading setting, analyzing how its performance changes once economic constraints are introduced and evaluating the gap between statistical asset pricing models and implementable investment strategies.

3 Data

Our analysis follows the data construction in (Bryzgalova et al.). We use monthly U.S. equity return data from CRSP over the sample period 1964–2016, along with firm characteristics constructed from CRSP/Compustat following the Kenneth French Data Library conventions. Excess returns are computed using the one-month Treasury bill rate as the risk-free rate.

For benchmarking, we include the S&P 500 index, proxied using adjusted close prices from Yahoo Finance, and a hedge fund long/short equity benchmark (CS L/S Equity Hedge Fund Main Index) obtained from [HFR](#). These benchmarks provide comparisons to both passive market exposure and actively managed market-neutral strategies.

4 Replication

4.1 Triple-Sort

Triple-sort is the natural baseline used in the reference work, being a conventional sorting method used in asset pricing literature. In this subsection, we describe the triple sort method for generating basis asset portfolios, replicating the work found in the paper and extending it to the investment-based approach considered in our own work.

In the triple-sort approach, we perform an *unconditional* sort on three stock-specific characteristics, such as size (log market equity), operating profitability, and investment. In particular, in the triple-sort 64 (TS64) approach, for each characteristic, stocks are sorted into quartiles, and portfolios are formed at the triple intersection of the buckets. For example, a representative portfolio may be formed from all those stocks that are in the first quartile of size, third quartile of operating profitability, and second quartile of investment. Notably, sorting as done in the reference work is performed *unconditionally*, i.e. the breakpoints between quantiles are formed independently for each characteristic before component stocks are bucketed into basis portfolios. Another version

of triple sort is the triple-sort 32 (TS32) approach, in which one characteristic (always size, in this work) is split only into two ($<$ or $>$ than median), creating 32, rather than 64 portfolios. Notably, such sorting-based methods have rapid (exponential) dimensionality growth as the number of characteristics used in the sort increases. As in the work, any cross sections considered are based on three characteristics, the first always being size and the second and third being two chosen out of the remaining 9 characteristics.

In triple sort, the portfolio construction step is not followed by separate portfolio selection, as the candidate portfolios are exactly those generated by the bucketed sort. We replicate the work of the authors by re-implementing the static optimization problem considered in the original work, which, given a set of candidate portfolios with sample mean and covariance $\hat{\mu}, \hat{\Sigma}$, along with a return target μ_0 , solves the regularized MVO problem,

$$\min_w \frac{1}{2} w^\top \hat{\Sigma} w + \lambda_1 \|w\|_1 + \frac{1}{2} \lambda_2 \|w\|_2, \text{ s.t. } w^\top \mathbf{1} = 1, w^\top \hat{\mu} \geq \mu_0. \quad (1)$$

4.2 AP Trees

The core insight from (Bryzgalova et al.) is that a sorted portfolio is essentially a nonparametric step function that estimates the conditional expected return surface in characteristic space. Triple sorts, for example, impose a rigid, pre-specified grid on three dimensions simultaneously. This rigidity creates two related problems. First, the grid cannot adapt to interaction effects: the value premium (B/M) is empirically concentrated among small-cap stocks, yet a triple sort applies the same value breakpoints uniformly across size quintiles. Second, the number of non-empty cells shrinks combinatorially as dimensions increase, making it infeasible to condition on more than three or four characteristics without pooling bins that may have very different return dynamics. The conditional tree structure fixes these issues.

The AP-Tree model recursively partitions the cross-section using a characteristic set $\mathcal{C}$. The tree has depth $D = 4$ and binary splits ($q = 2$).

Two structural constraints distinguish AP-Trees from standard CART. First, balanced node construction divides each node into equally populated quantile groups, ensuring that no leaf portfolio is empty or dominated by a handful of securities. Second, splits are applied sequentially within each existing node rather than independently, so that each partition is nested within the previous one. This conditional structure allows the model to capture interaction effects—for instance, allowing the OP breakpoint to vary across size groups—while maintaining interpretable, tradable leaf portfolios.

Each node portfolio is value-weighted: the weight of stock i in node portfolio p at time t is

$$w_{i,p,t}^{\text{stock}} = \frac{\text{ME}_{i,t}}{\sum_{j \in p_t} \text{ME}_{j,t}}, \quad (2)$$

and the candidate portfolio return is

$$x_{p,t+1} = \sum_{i \in p_t} w_{i,p,t}^{\text{stock}} r_{i,t+1}. \quad (3)$$

Rather than pruning by local prediction error as in CART, AP-Pruning retains a split only if it strictly increases the *global* maximum Sharpe ratio of the full portfolio set—a direct SDF spanning requirement. Portfolios that repackage existing risk exposures without adding non-redundant pricing information are eliminated, reducing dimension and limiting unnecessary turnover. Finally, the paper relaxes the assumption of a constant SDF projection by estimating optimal portfolio combination weights on a rolling 20-year window with shrinkage applied to both the mean vector and covariance matrix. This produces a time-adaptive tangency portfolio that consistently improves out-of-sample Sharpe ratios relative to the static benchmark.

Our replication follows the paper’s cross-section structure: we fix LME (log market equity, a proxy for size) as the primary sorting characteristic and interact it with pairs drawn from the remaining 9 characteristics, yielding $\binom{9}{2} = 36$ unique cross-sections, each evaluated using both the AP-Tree and Triple Sort methodologies.

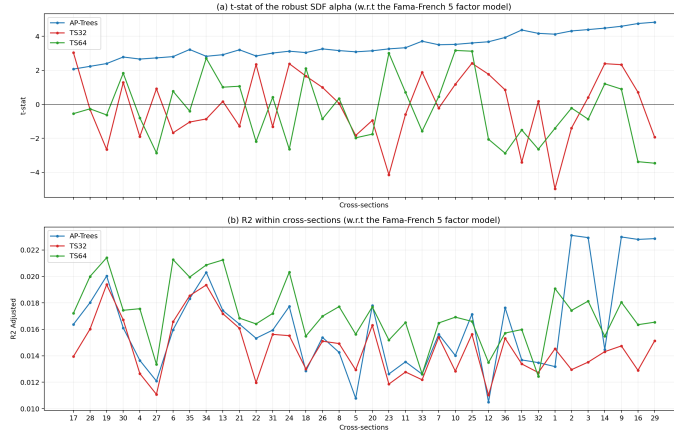


Figure 1: (a) t -stat of the robust SDF alpha and (b) R^2 -squared within cross-sections

Figure 1(a) reports the t -statistic of the mean SDF return across cross-sections. AP-Trees deliver statistically significant mean SDF returns in a larger fraction of cross-sections than Triple Sort. Importantly, the t -statistics from AP-Trees exhibit lower cross-sectional variance, consistent with the pruning procedure filtering out portfolios whose Sharpe contribution does not clear the statistical threshold. Figure 1(b) plots the cross-sectional R^2 from regressing realized returns on SDF-implied expected returns. AP-Trees achieve higher R^2

values than Triple Sort in the majority of cross-sections, with the gap widening as the number of characteristics increases. This pattern directly illustrates the motivation of the paper: conditional partitioning captures interaction effects in the return surface that a fixed triple-sort grid cannot recover.

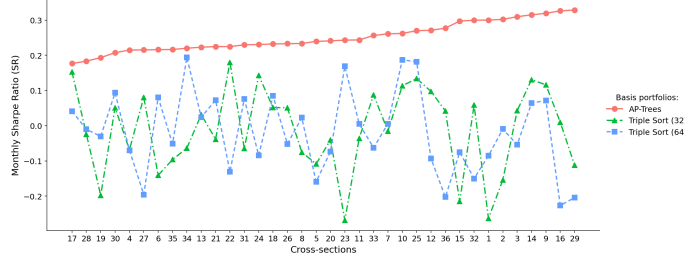


Figure 2: Sharpe ratios of SDFs spanned by pruned AP-Trees and triple sorts

Figure 2 plots, for each of the 36 cross-sections, the out-of-sample annualized Sharpe ratio of the AP-Tree SDF against the number of candidate portfolios retained after pruning. AP-Trees consistently achieve higher Sharpe ratios than Triple Sort benchmarks across cross-sections, with the advantage most pronounced in cross-sections where the two non-size characteristics exhibit strong interaction effects (e.g., $\text{LME} \times \text{BEME} \times \text{OP}$). The pruning is effective: the median tree retains substantially fewer than the theoretical maximum of leaf portfolios, confirming that redundant splits are systematically eliminated.

Our replication is consistent with the paper’s core findings. AP-Trees outperform Triple Sort along all three dimensions—Sharpe ratio, statistical significance, and cross-sectional fit—across the majority of the 36 cross-sections examined. The qualitative patterns in our figures match those reported in (Bryzgalova et al.) closely; remaining differences in scale are attributable to the privacy-perturbed nature of the published data rather than methodological discrepancies.

5 Model Extension

5.1 Transaction-cost-aware Rolling Optimization

Having replicated TS32, TS64, and AP-Trees, we extend the analysis to investable strategies by incorporating transaction costs. Even static strategies incur nontrivial costs due to monthly re-sorting and changing weights, limiting practical applicability.

We introduce a *rolling-window transaction cost-aware* optimization:

$$\min_{w_t} \quad \frac{1}{2} w_t^\top \hat{\Sigma}_t w_t - \eta \hat{\mu}_t^\top w_t + \frac{1}{2} \lambda_2 \|w_t\|_2^2 + \lambda_{\text{TC}} \text{TC}(w_t, w_{t-1}) \quad (4)$$

$$\text{TC}(w_t, w_{t-1}) = \|M_t w_t - M_{t-1} w_{t-1}\|. \quad (5)$$

Relative to the baseline, weights are time-varying and estimated using a 120-month rolling window for $(\hat{\mu}_t, \hat{\Sigma}_t)$. Return weighting is controlled by η , and transaction costs are imposed at the stock level via M_t , where $(M_t)_{ip}$ denotes the weight of stock i in portfolio p . Hyperparameters are fixed at $\eta = 0.15$, $\lambda_2 = 0.01$, and $\lambda_{TC} = 0.0025$. All experiments are conducted on the LME-OP-INV specification.

To obtain implementable portfolios, we convert SDF weights into trading weights by applying the depth adjustment and normalizing by gross exposure, ensuring $\|w\|_1 = 1$. This differs from the net-exposure normalization used for SDF evaluation, but is necessary for realistic backtesting and transaction cost analysis. Finally, we apply a one-period lag in portfolio construction to avoid lookahead bias.

5.1.1 Triple Sort Results

We now present our results for the triple sort approach shown in Fig 3, which we now describe here. In particular, we examine the cumulative net wealth of both TS32 and TS64 static and rolling TC-aware strategies, compared to the S&P 500, as a measure of performance against simple long-only strategies, and the CS US Equity HF index, as a measure of performance against long-short strategies. First, we observe that the inclusion of transaction costs diminishes the annualized Sharpe of the static strategy by roughly 20% in both cases, corresponding to the rapid movement of stocks between triple sort portfolios that corresponds to weight adjustments. Additionally, we notice that the inclusion of transaction-cost awareness has both a smaller Sharpe ratio and additional drawdown compared to the static strategy, although its performance on both metrics remains better than the S&P500.

5.1.2 AP Trees Results

As shown in Fig 4, the static AP-Tree variants exhibit extremely strong cumulative growth, reaching roughly $40\times$ (no-TC) and $32\times$ (static stock-TC) net wealth, while the rolling stock-TC variant compounds to approximately $15\times$; all three substantially outperform the S&P500 and hedge fund index, which end near $6\times$. The summary statistics show that the static variants achieve Sharpe ratios around 0.75/0.70 (gross/net), while the rolling variant reaches roughly 0.67/0.63, close to the S&P500 but below the hedge fund index (≈ 0.93). However, the max-drawdown bars indicate that this performance comes with substantial tail risk, with AP-Tree losses approaching -0.65 , worse than the S&P500 (≈ -0.51) and far deeper than the hedge fund index (≈ -0.22), suggesting that AP-Trees generate strong cumulative returns but with significantly higher downside risk.

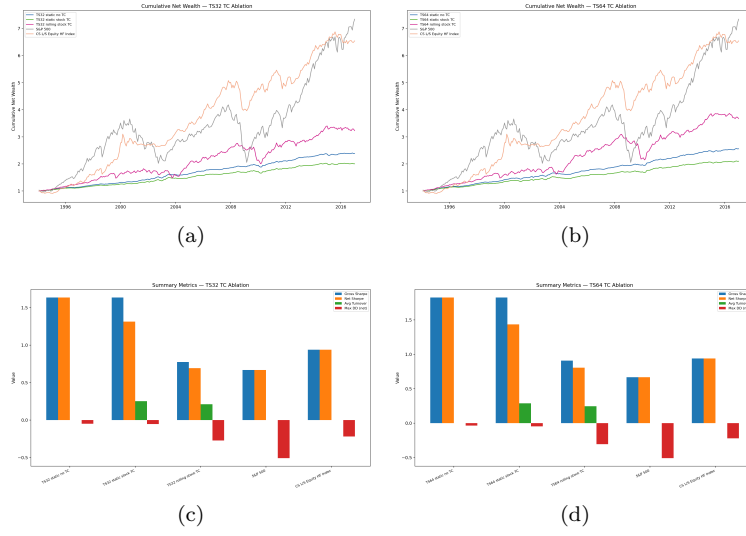


Figure 3: (Top row) Cumulative Net Returns (a) TS32 and (b) TS64 versus Benchmarks
(Bottom row) Performance Summary (c) TS32 and (d) TS64 versus Benchmarks

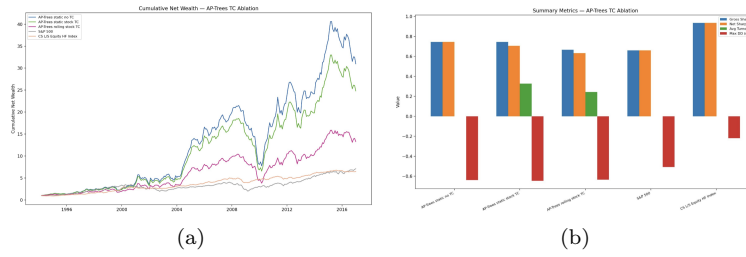


Figure 4: (a) Cumulative Net Returns (AP-Trees versus Benchmarks) (b) Performance Summary (AP-Trees versus Benchmarks)

5.2 AP Trees with Residual Momentum

We extend the AP-Tree framework by incorporating a residual momentum signal within each node. While AP-Trees group stocks based on firm characteristics, the baseline assigns value weights and does not exploit return dynamics within nodes. This extension introduces a tilt toward stocks with stronger idiosyncratic return persistence while preserving the tree structure.

5.2.1 Residual Momentum Construction

For each stock i , we estimate a rolling market model over a 36-month window:

$$r_{i,t} = \alpha_i + \beta_i r_{m,t} + \epsilon_{i,t}. \quad (6)$$

Residual momentum is defined as the cumulative product of residual returns from $t - 12$ to $t - 2$:

$$RM_{i,t} = \prod_{k=2}^{12} (1 + \hat{\epsilon}_{i,t-k}) - 1. \quad (7)$$

We standardize the signal cross-sectionally:

$$z_{i,t} = \frac{RM_{i,t} - \overline{RM}_t}{\sigma_{RM,t}}. \quad (8)$$

5.2.2 Within-Node Weighting

Let p_t denote the set of stocks in node p at time t . The baseline value weights are

$$w_{i,p,t}^{\text{stock}} = \frac{\text{ME}_{i,t}}{\sum_{j \in p_t} \text{ME}_{j,t}}. \quad (9)$$

We apply an exponential residual momentum tilt:

$$\tilde{w}_{i,p,t} = w_{i,p,t}^{\text{base}} e^{\tau z_{i,t}}, \quad w_{i,p,t}^{\text{RM}} = \frac{\tilde{w}_{i,p,t}}{\sum_{j \in \mathcal{N}_{p,t}} \tilde{w}_{j,p,t}}. \quad (10)$$

Node returns are then

$$R_{p,t} = \sum_{i \in \mathcal{N}_{p,t}} w_{i,p,t}^{\text{RM}} r_{i,t}. \quad (11)$$

5.2.3 Integration with AP-Trees

The residual-momentum-adjusted node returns replace the original value-weighted node returns in the AP-Tree candidate matrix. The tree construction and portfolio selection remain unchanged, so this ablation isolates the effect of modifying only the within-node weighting.

5.2.4 Transaction-cost-aware Rolling Optimization Results

As shown in Fig 5, AP-Trees + RM static strategies exhibit moderate growth, broadly tracking or slightly underperforming the S&P500 and hedge fund index, while the rolling stock-TC strategy dramatically outperforms, reaching roughly $16\text{--}17\times$ terminal wealth versus $\sim 6\text{--}7\times$ for benchmarks. This reflects the stronger cross-sectional signal from residual momentum, which the rolling optimizer exploits, but at the cost of higher risk, with a deeper drawdown compared to smoother static variants. In performance terms, static AP+RM achieves extremely high Sharpe (gross ≈ 2.2 , net ≈ 2.2 no-TC; ≈ 1.9 with TC), while the rolling strategy declines to $\approx 1.2/1.1$ due to transaction costs but remains above benchmarks. Turnover is significantly higher—especially for the rolling strategy—highlighting that residual momentum improves return predictability but introduces a clear trade-off through higher turnover, volatility, and cost sensitivity.

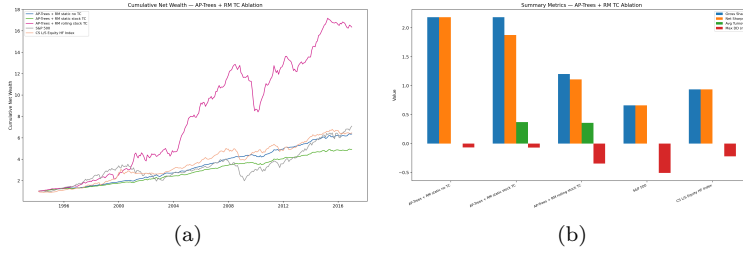


Figure 5: (a) Cumulative Net Returns (AP-Trees + RM versus Benchmarks)
(b) Performance Summary (AP-Trees + RM versus Benchmarks)

5.2.5 Combined Results with Transaction-cost-aware Rolling Optimization

Fig 6 shows that AP-Trees + RM delivers the strongest performance, reaching roughly $16\text{--}17\times$ terminal wealth, outperforming AP-Trees ($\sim 15\times$), triple sort ($\sim 3\text{--}4\times$), and benchmarks ($\sim 6\text{--}7\times$), but with higher variability and drawdowns. AP-Trees exhibits the deepest losses (up to ≈ -0.65), while AP+RM shows substantial but smaller drawdowns (≈ -0.30 to -0.35), both worse than benchmarks, whereas TS32 and TS64 remain smoother. In performance terms, AP+RM achieves the highest Sharpe (gross ≈ 1.2 , net ≈ 1.1) despite the highest turnover, followed by TS64 (comparable to the hedge fund index), TS32, and AP-Trees (Sharpe ≈ 0.65). Overall, results highlight a trade-off: AP-Trees relies on persistent structural exposure with high volatility, while AP+RM adds a stronger cross-sectional signal that improves risk-adjusted performance at the cost of higher turnover and drawdowns, whereas triple sort remains limited by weaker signals.

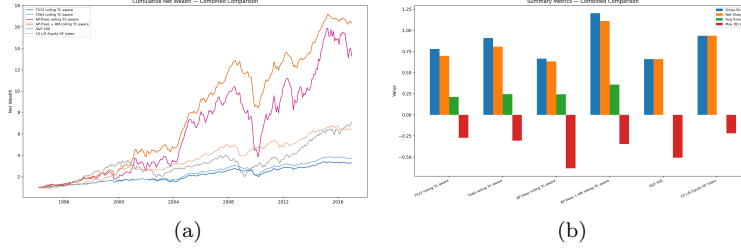


Figure 6: (a) Cumulative Net Returns (Model Comparison versus Benchmarks)
(b) Performance Summary (Model Comparison versus Benchmarks)

5.3 Conclusion

This paper replicates and extends the Forest Through the Trees framework of (Bryzgalova et al.), evaluating both its statistical performance and its implementability. We find that AP-Trees represent a clear improvement over traditional triple sorting by capturing nonlinear interactions and reducing redundancy through global pruning, resulting in more informative basis assets. However, their strong performance is partly driven by persistent structural exposure, and once realistic trading constraints are introduced, returns are meaningfully reduced. This highlights that statistical asset pricing performance does not directly translate into implementable strategies.

The residual momentum extension introduces a genuine cross-sectional alpha signal that improves performance, particularly under rolling transaction cost-aware optimization. However, this comes at the cost of higher turnover and greater sensitivity to trading frictions, reinforcing a fundamental trade-off between signal strength and implementability. Overall, our results show that while AP-based methods outperform traditional approaches, a significant portion of returns is still driven by implicit market and factor exposure rather than pure alpha, emphasizing the importance of incorporating realistic portfolio construction constraints.

Disclosure

AI tools were used for limited coding assistance; all modeling, design, and results were independently developed and validated by the authors.

References

- [1] Bryzgalova, Svetlana, et al. *Forest through the Trees: Building Cross-Sections of Stock Returns*. SSRN Electronic Journal, 2019. <https://doi.org/10.2139/ssrn.3493458>.